



FORMULARIO DE PRODUCCIÓN DE CONTENIDOS

NOMBRE DE LA ASIGNATURA: Desarrollo en plataformas
NOMBRE COMPLETO DEL DOCENTE CONTENIDISTA: Damián Aníbal Nicolalde Rodríguez

DESARROLLO

1. INTRODUCCIÓN DE LA CLASE (2 párrafos)

En esta tercera clase profundizaremos en los principios que permiten que una interfaz web sea flexible, escalable y adaptable a distintos dispositivos. Comprenderás cómo el modelo de caja (Box Model) define la estructura básica de todo elemento HTML y cómo su manipulación mediante propiedades de espaciado, bordes y márgenes influye directamente en la estética y funcionalidad de un sitio. Desde esta base, daremos el paso hacia los sistemas de organización modernos —Flexbox y CSS Grid— que reemplazan los antiguos métodos basados en flotaciones, ofreciendo control total sobre la disposición de los elementos tanto en una como en dos dimensiones.

Asimismo, exploraremos el concepto de diseño responsivo y el uso estratégico de Media Queries para crear experiencias coherentes en móviles, tabletas y escritorios. Analizaremos también el papel de los frameworks CSS contemporáneos (como Bootstrap y Tailwind) en la estandarización y aceleración del desarrollo, destacando sus ventajas y limitaciones frente al CSS puro. Al finalizar la clase, serás capaz de construir interfaces visualmente consistentes, accesibles y adaptables, integrando estructura, estilo y respuesta al contexto en una sola arquitectura visual.

Clase 3: CSS3 y diseño adaptable

RDA 1: Programar aplicaciones web dinámicas, interactivas y con diseño visualmente adaptable, que permita mejorar la experiencia del usuario.

Reto # 1

3.1. CSS3 y diseño adaptable

El **lenguaje CSS3** (Cascading Style Sheets nivel 3) es el estándar utilizado para definir la presentación visual de los documentos HTML. Su importancia radica en que permite separar la estructura del contenido (HTML) de la forma en que se muestra (CSS), facilitando la creación de sitios web modernos, accesibles y visualmente atractivos.

Uno de los conceptos más relevantes al trabajar con CSS3 es el diseño adaptable (adaptive design), que consiste en lograr que una interfaz pueda ajustarse a distintos



dispositivos y tamaños de pantalla de manera automática, sin sacrificar usabilidad ni estética.

1. ¿Qué es CSS3 y qué aportó frente a versiones anteriores?

CSS3 introdujo un conjunto de módulos independientes que ampliaron notablemente las capacidades del lenguaje. A diferencia de CSS2, que era más rígido, CSS3 permite evolucionar cada módulo por separado (tipografía, fondos, animaciones, transformaciones, layout, etc.).

Tabla 1. Comparación de CSS2 vs CSS3

Característica	CSS2	CSS3
Estructura	Monolítica, todas las reglas en un bloque	Modular, dividido en diferentes especificaciones
Tipografía	Limitada a fuentes del sistema	@font-face permite usar fuentes personalizadas
Layout	Flotantes y posiciones absolutas	Flexbox y Grid para layouts complejos
Colores	Paleta básica y hexadecimal	RGBA, HSLA, gradientes lineales y radiales
Animaciones	Necesitaban JavaScript	Nativas con <code>transition</code> y <code>@keyframes</code>

Fuente: elaboración propia (2025)

2. Principios del diseño adaptable

El diseño adaptable busca que un sitio web funcione y se vea bien en cualquier contexto, desde un celular pequeño hasta una pantalla 4K.



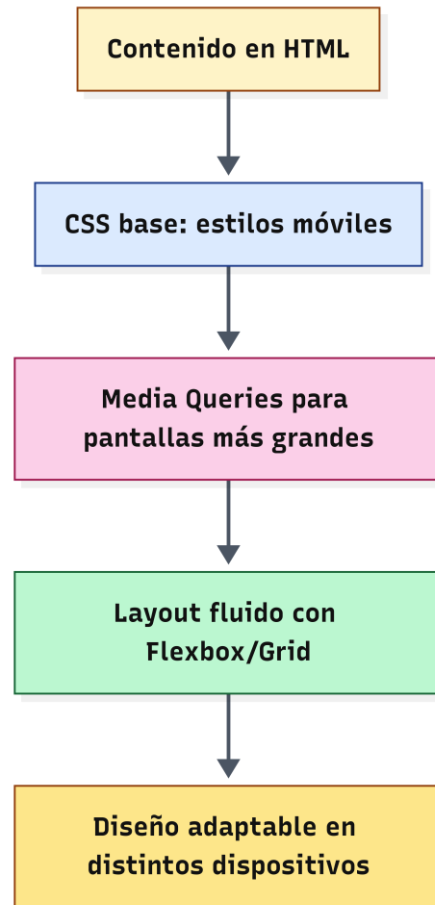


Imagen 1. Idea general del diseño adaptable

Fuente: creación propia (2025).

Principios fundamentales:

1. **Mobile-first:** Se diseña primero pensando en dispositivos pequeños y luego se agregan reglas para pantallas más grandes.
2. **Fluidez:** Se emplean unidades relativas (% , em , rem , vw , vh) en lugar de píxeles fijos.
3. **Imágenes y videos flexibles:** `max-width: 100%` garantiza que no desborden.
4. **Jerarquía tipográfica adaptable:** se utilizan escalas fluidas, por ejemplo:
5. `h1 { font-size: clamp(1.5rem, 2vw, 2.5rem); }`

Con esto, el tamaño mínimo es 1.5 rem, el máximo 2.5 rem y entre esos límites se ajusta al ancho de la ventana.

3. Diferencia entre diseño adaptable y diseño responsivo

Aunque suelen usarse como sinónimos, no son idénticos:

- **Adaptable:** utiliza diseños predefinidos que “saltan” entre versiones según el dispositivo (móvil, tableta, escritorio).
- **Responsivo:** ajusta el contenido de manera fluida y continua, reaccionando a cada variación en la ventana.

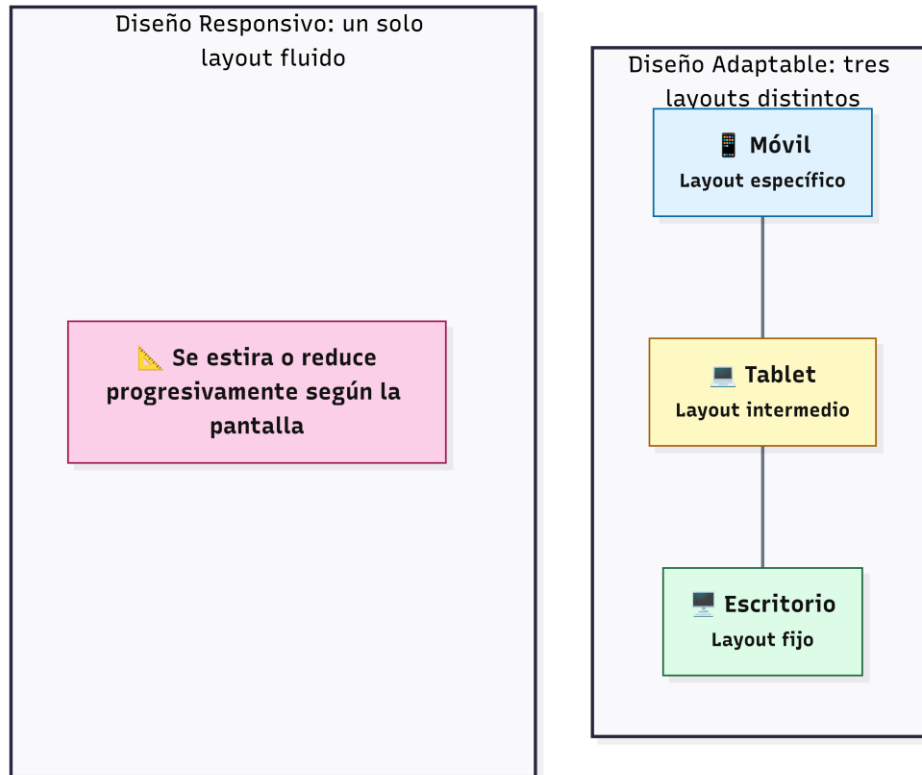


Imagen 2. Adaptable vs. responsivo

Fuente: elaboración propia (2025).

Título del enlace relacionado: Guía completa de diseño web responsivo (MDN Web Docs)

Descripción del enlace relacionado: Recurso oficial de Mozilla Developer Network que explica de manera práctica cómo aplicar CSS3 y Media Queries para construir sitios web responsivos. Incluye ejemplos y buenas prácticas.

Enlace: https://developer.mozilla.org/es/docs/Learn/CSS/CSS_layout/Responsive_Design



4. Unidades relativas y su rol en la adaptabilidad

Uno de los errores comunes en principiantes es fijar medidas en píxeles. CSS3 recomienda trabajar con unidades relativas, que permiten escalabilidad.

Tabla 2. Unidades más comunes

Unidad	Referencia	Ejemplo	Uso recomendado
%	Elemento contenedor	<code>width: 80%</code>	Layouts fluidos
em	Tamaño de la fuente del elemento	<code>padding: 2em</code>	Espaciados relativos a texto local
rem	Tamaño de la fuente raíz	<code>font-size: 1.2rem</code>	Tipografía consistente
vw	1% del ancho de la ventana	<code>font-size: 5vw</code>	Texto que crece con viewport
vh	1% de la altura de la ventana	<code>height: 50vh</code>	Secciones que llenan pantalla

Fuente: elaboración propia (2025).

5. Buenas prácticas para un diseño adaptable

1. Evitar anchos fijos:

```
contenedor { max-width: 1200px; margin: auto; }
```

Esto asegura que el contenido nunca sea más ancho de lo necesario.

2. Hacer que las imágenes sean flexibles:

```
img { max-width: 100%; height: auto; }
```

3. Diseñar sistemas de espaciado:

Usar variables CSS:

```
:root {
  --space-1: 0.5rem;
  --space-2: 1rem;
  --space-3: 2rem;
}
```

4. Mantener la legibilidad:

Líneas de 60–80 caracteres, `line-height: 1.5;`, y contraste suficiente entre texto y fondo.



```
body {  
  font-family: sans-serif;  
  margin: 0 auto;  
  max-width: 1200px;  
  padding: 1rem;  
}  
  
h1 {  
  font-size: clamp(1.5rem, 2vw, 2.5rem);  
}
```

Imagen 3. Ejemplo de layout fluido básico

Fuente: elaboración propia (2025).

6. Errores frecuentes

- **Medidas rígidas:** usar `width: 960px;` en vez de `max-width`.
- **Olvidar probar en distintos dispositivos:** diseñar solo en un monitor grande.
- **No usar media queries:** confiar en que el navegador “lo ajustará solo”.
- **Abusar de píxeles:** dificulta la escalabilidad.
- **Ignorar accesibilidad:** contraste insuficiente o texto muy pequeño.

7. Caso práctico narrado

Situación: un estudiante diseña una página de recetas con un título, imagen y lista de ingredientes.

1. En escritorio, todo se ve perfecto: imagen grande a la izquierda, ingredientes a la derecha.
2. En celular, la imagen ocupa toda la pantalla y los ingredientes se pierden abajo.
3. Con CSS3 adaptable, se corrige:
 - Se establece `max-width: 100%` para la imagen.
 - Se define un layout con `display: flex` que pasa de columnas en escritorio a filas en móvil usando `@media (max-width: 600px)`.

Resultado: en cualquier dispositivo, la receta se lee con claridad.

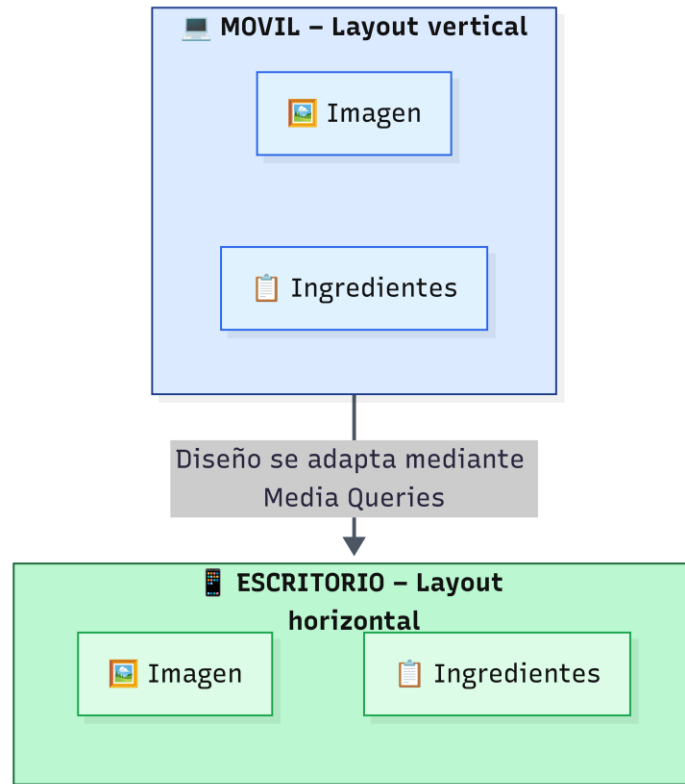


Imagen 4. Caso práctico de adaptación

Fuente: elaboración propia (2025).

8. Resumen

El diseño adaptable con CSS3 no es un lujo, sino una necesidad en la web actual. Un usuario puede entrar desde un celular, una tableta, una laptop o un televisor, y la interfaz debe responder de manera óptima en todos los casos.

CSS3 proporciona las herramientas necesarias: unidades relativas, tipografía fluida, imágenes flexibles, Media Queries, Flexbox y Grid, además de prácticas de accesibilidad.

3.2. Box model, Flexbox y Grid

La forma en que los navegadores renderizan los elementos HTML depende de un concepto central: el modelo de caja (box model). Cada etiqueta en una página se interpreta como una caja rectangular que tiene contenido, relleno, borde y márgenes.

CSS3 introduce herramientas más modernas para organizar estas cajas en layouts complejos: **Flexbox** y **Grid**, que reemplazan a técnicas antiguas como los `float` o el uso excesivo de tablas.

1. El modelo de caja (Box model)

Cada elemento de una página web está envuelto en un "box". Comprender este modelo es esencial para controlar el espacio, las alineaciones y los diseños.

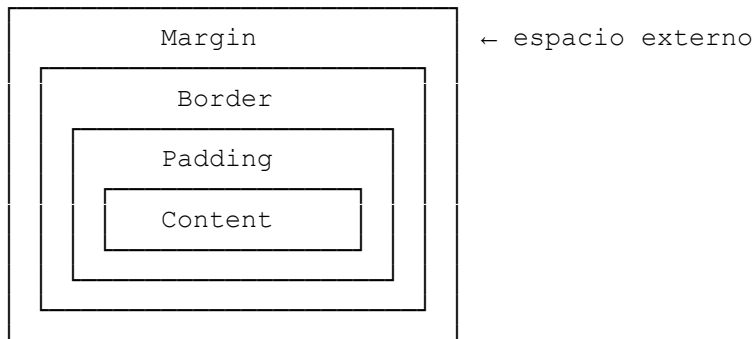


Imagen 5. Esquema del modelo de caja

Fuente: elaboración propia (2025).

Componentes principales:

1. **Content:** el texto, imagen o elemento principal.
2. **Padding:** espacio interno entre el contenido y el borde.
3. **Border:** línea que rodea el padding y contenido.
4. **Margin:** espacio externo, separa la caja de otras cajas.

Ejemplo en CSS:

```
.caja {
  width: 200px;
  padding: 20px;
  border: 2px solid black;
  margin: 10px;
}
```

Este bloque tendrá:

- 200px de ancho de contenido.
- +20px de padding en cada lado.
- +2px de borde.
- +10px de margen alrededor.

Ancho real total = 200 + (20x2) + (2x2) = 244px.

Para simplificar, se suele usar:

```
* { box-sizing: border-box; }
```



Así el ancho total incluye padding y borde, evitando cálculos manuales.

2. Flexbox: diseño unidimensional

Flexbox (Flexible Box Layout) organiza cajas en una dirección principal: fila o columna. Es ideal para alinear elementos dentro de un contenedor.

```
Fila (row):    [Caja 1][Caja 2][Caja 3]

Columna (col):

[Caja 1]
[Caja 2]
[Caja 3]
```

Imagen 6. Idea de Flexbox

Fuente: elaboración propia (2025).

Propiedades clave del contenedor flex:

```
.contenedor {
  display: flex;
  flex-direction: row; /* o column */
  justify-content: space-between; /* distribución horizontal */
  align-items: center; /* alineación vertical */
}
```

- **display: flex:** activa Flexbox.
- **flex-direction:** eje principal (row o column).
- **justify-content:** organiza en el eje principal (start, center, end, space-between, space-around).
- **align-items:** organiza en el eje cruzado (stretch, center, flex-start, flex-end).

Propiedades de los hijos flex:

```
.item {
  flex: 1; /* ocupa mismo espacio que los demás */
}
```

Tabla 1. Justify-content vs Align-items

Propiedad	Función	Ejemplo visual
justify-content	Ordena horizontalmente	[A B C]
align-items	Ordena verticalmente	A B C

Fuente: elaboración propia (2025)



```
header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```

Imagen 7. Ejemplo práctico Flexbox

Fuente: elaboración propia (2025).

Resultado: el logo queda a la izquierda y el menú de navegación a la derecha, perfectamente alineados.

3. Grid: diseño bidimensional

Mientras que Flexbox organiza elementos en una sola dimensión (fila o columna), CSS Grid Layout lo hace en dos dimensiones (filas y columnas simultáneamente).

A	B	C	← Fila 1
D	E	F	← Fila 2

Imagen 8. Idea de Grid

Fuente: elaboración propia (2025).

Propiedades del contenedor grid:

```
.grid {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr; /* 3 columnas */  
  grid-template-rows: auto 200px; /* 2 filas */  
  gap: 10px;  
}
```

- **display: grid:** activa Grid.
- **grid-template-columns / rows:** define estructura.
- **fr:** unidad fraccional (divide espacio disponible).
- **gap:** separa filas y columnas.

Propiedades de los hijos grid:

```
.item-a {  
  grid-column: 1 / 3; /* ocupa 2 columnas */  
}  
.item-b {  
  grid-row: 1 / 3; /* ocupa 2 filas */  
}
```



Tabla 2. Flexbox vs Grid

Característica	Flexbox	Grid
Dirección	Una (fila o columna)	Dos (filas y columnas)
Complejidad	Más simple	Más potente
Uso típico	Menús, barras, alineación de items	Tableros, layouts completos de página

Fuente: elaboración propia (2025).

4. Ejemplo integrador

Código:

```
body {
  display: grid;
  grid-template-columns: 1fr 3fr;
  grid-template-rows: auto 1fr auto;
  gap: 10px;
  min-height: 100vh;
}

header {
  grid-column: 1 / 3;
  background: lightblue;
}

aside {
  background: lightgray;
}

main {
  background: white;
}

footer {
  grid-column: 1 / 3;
  background: lightblue;
}
```

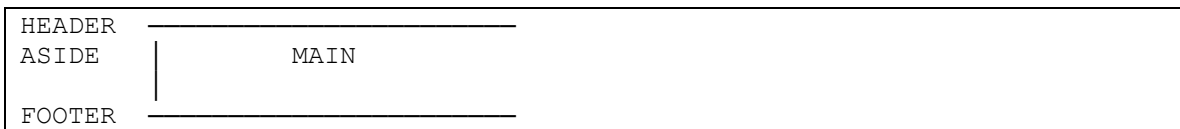


Imagen 9. Layout con Grid

Fuente: elaboración propia (2025).

5. Buenas prácticas

1. Usar Flexbox para alineación local (por ejemplo, elementos dentro de un `header`).
2. Usar Grid para estructura global (por ejemplo, toda la página).



3. Definir breakpoints con media queries para reorganizar el Grid en móviles.
4. Combinar ambos: Grid para macroestructura + Flexbox para microdetalles.

6. Resumen

- El Box Model es la base: todo elemento HTML es una caja con contenido, padding, borde y margen.
- Flexbox simplifica la alineación y distribución en una dimensión.
- Grid permite organizar elementos en dos dimensiones y construir layouts completos.
- En la práctica: usa Grid para la estructura global y Flexbox dentro de cada sección.

Un dominio claro de estas herramientas permite pasar de páginas rígidas a interfaces flexibles, escalables y elegantes, requisito esencial en el desarrollo de plataformas modernas.

3.3. Media Queries y diseño responsivo

El diseño responsivo es la adaptación fluida y progresiva de una interfaz web a distintos tamaños y características de dispositivos. Su herramienta principal en CSS3 son las Media Queries, que permiten aplicar reglas de estilo condicionales dependiendo de propiedades del dispositivo o del navegador.

Esto asegura que una página web se vea bien en un celular de 5 pulgadas, en una tablet de 10 pulgadas, en un monitor de escritorio o incluso en un televisor 4K.

1. Concepto de Media Query

Una *Media Query* es una instrucción en CSS que define condiciones (queries) sobre el entorno de visualización. Si esas condiciones se cumplen, se aplican ciertos estilos.

Sintaxis básica

```
@media (condición) {  
  /* Estilos que se aplican solo si la condición es verdadera */  
}
```

Ejemplo:

```
@media (max-width: 600px) {  
  body {  
    background-color: lightblue;  
  }  
}
```



Cuando el ancho de la pantalla es 600px o menos, el fondo de la página cambia a celeste.

2. Principio de Mobile First

En el diseño moderno se recomienda comenzar con estilos para móviles (pantallas pequeñas) y luego agregar reglas progresivas para pantallas más grandes.

Esto se logra usando min-width:

```
/* Estilos base: móvil */
body { font-size: 16px; }

/* Tablet y superiores */
@media (min-width: 768px) {
  body { font-size: 18px; }
}

/* Escritorio y superiores */
@media (min-width: 1200px) {
  body { font-size: 20px; }
}
```

ESTILOS BASE → min-width:768px → min-width:1200px
(Móvil) (Tablet) (Desktop)

Imagen 10. Esquema Mobile First

Fuente: elaboración propia (2025).

3. Propiedades comunes en Media Queries

Los navegadores permiten condicionar estilos en base a muchas características:

Tabla 1. Propiedades comunes en Media Queries

Propiedad	Descripción	Ejemplo
width / max-width	Ancho de la ventana	(max-width: 600px)
height / max-height	Alto de la ventana	(min-height: 800px)
orientation	Orientación del dispositivo	(orientation: landscape)
aspect-ratio	Relación ancho/alto	(min-aspect-ratio: 16/9)
prefers-color-scheme	Tema claro u oscuro del usuario	(prefers-color-scheme: dark)
prefers-reduced-motion	Preferencia de animaciones reducidas	(prefers-reduced-motion: reduce)

Fuente: elaboración propia (2025).

4. Ejemplo práctico de Media Queries

Código

```
.card {
  background: white;
  padding: 1rem;
  margin: 1rem;
}

/* Tablets */
@media (min-width: 768px) {
  .card {
    display: flex;
    flex-direction: row;
  }
}

/* Escritorio */
@media (min-width: 1200px) {
  .card {
    max-width: 1000px;
    margin: 2rem auto;
  }
}
```

En móviles, cada tarjeta se apila en columna.

En tablets, las tarjetas usan `flex-row`.

En escritorio, se centran con un ancho máximo.

```
MÓVIL:
[ Tarjeta ]
[ Tarjeta ]

TABLET:
[ Tarjeta | Tarjeta ]

ESCRITORIO:
[ Tarjeta | Tarjeta | Tarjeta ]
```

Imagen 11. Adaptación de tarjetas con Media Queries

Fuente: elaboración propia (2025).

5. Breakpoints comunes

No hay reglas universales, pero existen convenciones de la industria que sirven como referencia inicial.

Tabla 2. Breakpoints sugeridos

Dispositivo	Ancho	Media Query recomendada
Móvil pequeño	0-480px	@media (max-width: 480px)
Móvil grande	481-767px	@media (min-width: 481px)
Tablet	768-1023px	@media (min-width: 768px)
Laptop	1024-1199px	@media (min-width: 1024px)
Desktop	1200px+	@media (min-width: 1200px)

Fuente: elaboración propia (2025).

6. Media Queries avanzadas

CSS3 va más allá del tamaño de pantalla e incluye preferencias del usuario:

1. Tema oscuro/claro:

```
@media (prefers-color-scheme: dark) {
  body { background: #121212; color: white; }
}
```

2. Reducción de movimiento:

```
@media (prefers-reduced-motion: reduce) {
  * { animation: none !important; }
}
```

Esto respeta la accesibilidad y evita mareos en personas sensibles.

```
LIGHT MODE: fondo blanco, texto negro
DARK MODE:  fondo negro, texto blanco
```

Imagen 12. Ejemplo de tema claro/oscuro con Media Queries

Fuente: elaboración propia (2025).

7. Buenas prácticas en diseño responsivo

1. No uses demasiados breakpoints: elige los necesarios, basados en contenido, no en dispositivos específicos.
2. Empieza simple: diseña un flujo vertical para móvil, luego mejora en pantallas más grandes.
3. Usa unidades relativas: rem, %, fr, vw escalan mejor que px.
4. Prueba en distintos dispositivos: simuladores y navegadores ofrecen herramientas para inspeccionar tamaños.
5. Piensa en accesibilidad: Media Queries como prefers-reduced-motion son tan importantes como las de tamaño.

8. Caso: menú de navegación

- En móvil, el menú aparece oculto en un botón “hamburguesa”.
- En tablet, los ítems se muestran en fila pero con texto pequeño.
- En escritorio, el menú ocupa toda la barra superior con espaciado amplio.

Código

```
nav ul {
  display: none;
}

/* Tablet */
@media (min-width: 768px) {
  nav ul {
    display: flex;
    gap: 1rem;
  }
}

/* Desktop */
@media (min-width: 1200px) {
  nav ul {
    gap: 2rem;
    font-size: 1.2rem;
  }
}
```

MÓVIL: ≡
 TABLET: [Home] [Blog] [Contacto]
 DESKTOP: [Home | Blog | Contacto]

Imagen 13. Menú responsivo con Media Queries

Fuente: elaboración propia (2025).

9. Resumen

- El diseño responsivo asegura que las páginas se adapten a todos los dispositivos.
- Media Queries son la base de esta adaptabilidad, permitiendo condicionar estilos según ancho, alto, orientación o incluso preferencias del usuario.
- La estrategia Mobile First es la más recomendada.
- Al aplicar buenas prácticas, los proyectos se vuelven más inclusivos, accesibles y escalables.

3.4. Frameworks CSS modernos: Bootstrap y Tailwind

En la actualidad, el desarrollo de interfaces web no solo depende de conocer CSS3 puro, sino también de aprovechar frameworks CSS modernos, que aceleran el trabajo y garantizan coherencia visual.

Dos de los más utilizados son:

- **Bootstrap**, creado en 2011 por desarrolladores de Twitter.
- **Tailwind CSS**, surgido en 2017 con un enfoque *utility-first* (clases pequeñas reutilizables).

Ambos permiten crear sitios adaptables, accesibles y con buenas prácticas de diseño, pero tienen filosofías distintas.

1. ¿Qué es un framework CSS?

Un framework CSS es un conjunto predefinido de reglas, componentes y utilidades que simplifica el diseño de páginas web.

Ventajas principales:

- Ahorra tiempo: ya trae botones, formularios, rejillas y tipografía.
- Consistencia: mantiene un estilo uniforme en todo el proyecto.
- Adaptabilidad: casi todos incluyen un sistema responsivo listo.

2. Bootstrap: rejilla y componentes listos

Bootstrap es el framework más popular del mundo, usado en millones de sitios. Se centra en ofrecer una rejilla responsiva (grid system) y una gran cantidad de componentes preconstruidos.

12 columnas → puedes combinar: [4 4 4] → tres iguales [6 6] → dos iguales [3 9] → una pequeña y otra grande
--

Imagen 14. Esquema de la rejilla de Bootstrap

Fuente: elaboración propia (2025).

Ejemplo de uso de la rejilla

```
<div class="container">
  <div class="row">
    <div class="col-4">Columna 1</div>
    <div class="col-4">Columna 2</div>
    <div class="col-4">Columna 3</div>
  </div>
</div>
```

Con pocas clases, se logra un layout adaptable.

Tabla 1. Ventajas y desventajas de Bootstrap

Ventajas	Desventajas
Gran comunidad y documentación	Puede sentirse rígido si no se personaliza
Componentes listos: menús, botones, modales	CSS algo pesado si no se optimiza
Rejilla responsiva fácil de usar	El diseño puede lucir genérico si no se adapta

Fuente: elaboración propia (2025).

3. Tailwind CSS: utilidades primero

A diferencia de Bootstrap, Tailwind CSS no trae componentes listos. Su enfoque es utility-first, es decir, ofrece clases pequeñas y específicas para estilos (colores, márgenes, tipografía, etc.), que el desarrollador combina para crear interfaces personalizadas.

Ejemplo en Tailwind

```
<div class="max-w-sm mx-auto bg-white rounded-xl shadow-md overflow-hidden p-4">
  <h1 class="text-xl font-bold text-gray-900">Título</h1>
  <p class="text-gray-600">Este es un párrafo con Tailwind.</p>
</div>
```

Aquí no hay CSS adicional: todo se define con clases.

Bootstrap:	<code><div class="card"> ... </div></code>
Tailwind:	<code><div class="bg-white p-4 shadow-md rounded-lg"> ... </div></code>

Imagen 15. Filosofía Utility-first

Fuente: elaboración propia (2025).

Tabla 2. Ventajas y desventajas de Tailwind

Ventajas	Desventajas
Gran flexibilidad, diseños únicos	Curva de aprendizaje inicial
Código más ligero (si se usa purge)	Puede verse sobrecargado de clases
Excelente integración con React, Vue, etc.	No trae componentes listos



Fuente: elaboración propia (2025).

4. Comparación entre Bootstrap y Tailwind

Bootstrap → Componentes listos (rápido)
Tailwind → Estilos utilitarios (flexible)

Imagen 16. Comparación visual

Fuente: elaboración propia (2025).

Tabla 3. Bootstrap vs Tailwind

Característica	Bootstrap	Tailwind
Filosofía	Componentes predefinidos	Clases utilitarias
Velocidad inicial	Muy rápida	Requiere aprender combinación de clases
Personalización	Limitada sin sobrescribir	Muy alta
Peso inicial	Mayor	Menor (con purge)
Comunidad	Muy amplia	En rápido crecimiento

5. Ejemplo comparativo

Bootstrap

```
<button class="btn btn-primary">Enviar</button>
```

Tailwind

```
<button class="bg-blue-500 hover:bg-blue-700 text-white font-bold py-2
px-4 rounded">
  Enviar
</button>
```

Bootstrap ofrece un botón listo, mientras que Tailwind permite definirlo con clases utilitarias.

6. Buenas prácticas con frameworks

1. No depender al 100%: aprende primero CSS puro.
2. Personaliza: cambia colores, tipografías y variables para que tu sitio no se vea genérico.
3. Optimiza: en producción, elimina clases o componentes que no uses.
4. Accesibilidad: revisa contraste, foco visible y compatibilidad con lectores de pantalla.
5. Combina lo mejor: puedes usar Tailwind para la base y Bootstrap para algunos componentes si el proyecto lo permite.



7. Caso: página de inicio

Un estudiante debe diseñar una landing page con un encabezado, un bloque central y un pie de página.

- Con **Bootstrap**, puede usar `container`, `row` y `col` para organizar el layout, y añadir un `navbar` ya construido.
- Con **Tailwind**, define `grid`, `gap-4`, `text-center` y colores directamente en las clases HTML, logrando un estilo único.

Bootstrap:	Layout estándar con componentes prehechos
Tailwind:	Layout único con clases utilitarias

Imagen 17. Resultado visual

Fuente: elaboración propia (2025).

8. Resumen

- Los frameworks CSS modernos son herramientas que aceleran el diseño.
- **Bootstrap** es ideal para empezar rápido, con componentes listos y una rejilla responsiva.
- **Tailwind** ofrece máxima flexibilidad con un enfoque utility-first, favoreciendo la personalización.
- La elección depende del proyecto: rapidez y estandarización con Bootstrap, o flexibilidad y unicidad con Tailwind.

Título del enlace relacionado: Bootstrap vs Tailwind: comparación práctica (CSS Tricks)

Descripción del enlace relacionado: Artículo que compara los frameworks CSS más populares, Bootstrap y Tailwind, destacando sus diferencias en filosofía, ventajas y usos prácticos en proyectos modernos.

Enlace: <https://css-tricks.com/tailwind-vs-bootstrap/>

Dominar estos frameworks es fundamental para un desarrollador moderno, pero siempre recordando que la base sigue siendo **CSS3 puro**.

Referencias citadas en la Clase 3.

- Frain, B. (2020). Responsive Web Design with HTML5 and CSS. Packt Publishing.
- Cederholm, D. (2018). CSS Grid Layout. A Book Apart.
- W3C. (2023). CSS Cascading Style Sheets Level 3 (CSS3) Specification. <https://www.w3.org/TR/css-2023/>



- Mozilla Developer Network (MDN). (2024). CSS: Cascading Style Sheets. <https://developer.mozilla.org/es/docs/Web/CSS>
- W3C. (2018). Media Queries Level 4. <https://www.w3.org/TR/mediaqueries-4/>.

Definición de términos

- **Box model**: concepto fundamental de CSS que describe cómo los navegadores calculan el espacio que ocupa cada elemento HTML, considerando contenido, padding, borde y márgenes.
 - **Media Query**: regla condicional en CSS3 que aplica estilos en función de características del dispositivo (ancho, alto, orientación, preferencias de usuario), permitiendo el diseño responsivo.
- Recursos de profundización

Recurso: *Profundizacion1_clase3.docx*